ITA0615 – MACHINE LEARNING
CO4 AT3

By:
Harshith G
192525394

1: K-NEAREST NEIGHBOUR (KNN) VS CASE-BASED LEARNING (CBL) – MEDICAL DIAGNOSIS

Introduction:

Both KNN and Case-Based Learning are instance-based (lazy) methods: they store past examples and solve a new problem by finding similar earlier ones, without building an explicit global model. KNN predicts by a vote of the K nearest points in a fixed feature space, whereas Case-Based Learning (Case-Based Reasoning) retrieves similar cases with rich structure and then reuses and adapts their solution.

Example:

Example – medical diagnosis: a hospital stores records of past patients. For a new patient, KNN converts the symptoms and test values (age, temperature, blood pressure, glucose) into a numeric vector, finds the K = 5 most similar patients using Euclidean distance and predicts the disease that most of them had. A CBL system stores each patient as a case (symptoms, test reports, history, diagnosis, treatment given and outcome), retrieves the most similar earlier case(s), reuses the diagnosis and treatment, revises them for this patient (e.g. changes the dose for age or allergy) and retains the new case after the doctor confirms it.

Comparison:

Aspect	**K-Nearest Neighbour (KNN)**	**Case-Based Learning (CBL)**
Memory usage	Stores all training samples as simple numeric vectors; memory grows linearly with data (O(n·d)). No extra information is stored.	Stores complete cases (problem + solution + outcome, often with text and symbolic data), so each case needs more memory; the case base is maintained by deleting redundant or outdated cases and by indexing.
Similarity measure	Single global distance on numeric features: Euclidean, Manhattan or Minkowski; all features usually have equal weight (after scaling).	Flexible, knowledge-based similarity: $Sim = \Sigma\, w_i \cdot sim_i$, with different local measures for numeric, categorical and symbolic attributes and feature weights given by experts.
Prediction strategy	Majority vote (classification) or mean (regression) of the K nearest neighbours; no adaptation of the neighbour's answer.	Retrieve → Reuse → Revise → Retain: the retrieved solution is adapted to the new problem and the new solved case is added to the case base.

Learning / new data	New samples are simply appended.	New cases are retained after validation; case-base maintenance is needed.
Output / explanation	Class label (and vote proportion).	A complete solution (diagnosis + treatment) with the similar past case as explanation.

Analysis:

- KNN needs less design effort and works well when data are numeric and plentiful, but it cannot use knowledge such as medical rules and gives only a label.
- CBL needs more effort (case structure, similarity and adaptation rules) but supports complex, partly symbolic problems and gives explainable, reusable solutions, which doctors prefer.
- Both suffer from slow retrieval on large data (search through all stored items) and both are sensitive to irrelevant features; indexing (KD-tree, hashing) is used by both.

Conclusion:

KNN is a simple special case of case-based reasoning. KNN is preferred for quick numeric classification, while Case-Based Learning is preferred for medical diagnosis where cases are structured, solutions must be adapted and the reasoning must be explained.

2: LOCALLY WEIGHTED REGRESSION (LWR) VS RADIAL BASIS FUNCTION (RBF) NETWORKS – FUNCTION APPROXIMATION

Introduction:
Both methods approximate a non-linear function using local information around the query point. LWR is a lazy, memory-based method that fits a new weighted linear model for every query using the training data. An RBF network is an eager method: a fixed number of Gaussian hidden units are trained once and the function is represented as a weighted sum of them.

Example:
Example dataset: 40 noisy points from $y = \sin(x) + \text{noise}$, $0 \le x \le 6$. LWR with a Gaussian kernel ($\tau = 0.5$) fits a straight line weighted around each query x_q, using $w(i) = \exp(-(x(i) - x_q)^2 / 2\tau^2)$, and predicts $\hat{y}(x_q)$ from that line. An RBF network with six Gaussian centres (e.g. found by K-Means) and width $\sigma = 1$ computes $\hat{y}(x) = \Sigma\, w_j \exp(-(x - c_j)^2 / 2\sigma^2) + b$, where the weights are found once by least squares. In a test run of this kind, both followed the sine curve closely (MSE ≈ 0.005 – 0.01), whereas a single linear regression had MSE ≈ 0.19.

Comparison:

Aspect	Locally Weighted Regression	RBF Network
Locality	Fully local and re-fitted for each query: only the training points near x_q get a significant weight; the local region moves with the query.	Local through the Gaussian units: each hidden unit responds only near its centre, but the centres are fixed after training and the output weights are global.
Training	No training; all data are stored (lazy learning).	Training needed: choose centres (K-Means), widths and solve output weights (least squares).
Computational cost	Cheap training but expensive prediction: a weighted least-squares problem (matrix inversion) per query, $O(n \cdot d^2)$ or worse; needs all n data points at prediction time.	Training cost higher once (clustering + least squares) but prediction is very fast, $O(m \cdot d)$, where m (number of centres) $\ll$ n; only centres and weights are stored.
Smoothness of predictions	Smooth if the bandwidth τ is suitably large; can become jagged / unstable when τ is small or data are sparse near the query.	Smooth and continuous by construction (sum of Gaussians); smoothness is controlled by the width σ and the number of centres.
Parameters	Bandwidth τ (and kernel type).	Number of centres m, centre positions, widths σ, regularisation.

New data	Added instantly.	Needs retraining or an online update.

Analysis:

- Small τ (LWR) or small σ (RBF) gives high variance and over-fitting; large values give over-smoothing and under-fitting, so both are tuned by cross-validation.
- LWR is best for small or medium data sets and when data keep changing; the RBF network is better for large data sets and real-time use, because its prediction cost is independent of n.

Conclusion:

Both approximate non-linear functions with similar accuracy. LWR gives flexible local fits at a high prediction cost, while the RBF network compresses the data into a few Gaussian units and gives fast, smooth predictions after a one-time training effort.

3: K-NEAREST NEIGHBOUR (KNN) VS RBF NETWORK – CLASSIFICATION

Introduction:
KNN classifies a point by looking directly at the labels of its nearest stored training points. An RBF network first learns a small set of Gaussian hidden units (prototypes) and a linear output layer that maps their activations to class scores.

Example:
Example: classifying two interleaving half-moon shaped classes in 2-D (300 training and 100 test points). KNN with K = 5 stores all 300 points and votes among the five nearest. An RBF network with 10 Gaussian centres (found by K-Means) and output weights found by least squares computes one score per class and chooses the largest. In a test of this kind both reached about 98% accuracy, but their training time, prediction time and boundaries differed as shown below.

Comparison:

Aspect	**KNN**	**RBF Network**
Decision boundary	Non-parametric and irregular (piece-wise, follows individual points); small K gives a jagged boundary, large K a smoother one.	Smooth, continuous boundary formed by combining Gaussian bumps; its flexibility depends on the number of centres and width σ.
Training time	None (lazy learner) – only stores the data.	Required: clustering for centres plus least squares for weights (a few milliseconds here), but much faster than back-propagation networks.
Prediction time / memory	Slow: distance to all n stored points for every query; memory $O(n \cdot d)$.	Fast: distance to only m centres ($m \ll n$); stores centres and weights.
Generalisation	Good with plenty of data; sensitive to noise, irrelevant features and the choice of K; suffers from the curse of dimensionality.	Good because the model summarises the data and is regularised; depends on a proper choice of centres and widths – too many centres over-fit, too wide units under-fit.
Output	Class label and vote fraction (e.g. 0.40).	Continuous class scores (e.g. 0.39 vs 0.61).
Parameters	K, distance measure.	Number of centres, widths, regularisation.

Analysis:

- On a boundary point both can differ, since KNN decides by the few nearest neighbours, while the RBF network decides from the combined influence of all centres.
- KNN is the better choice when the data set is small, changes often and training must be avoided; the RBF network is better for large data sets and real-time prediction.

Conclusion:

Both can model non-linear class boundaries. KNN is simple but costly at prediction time, whereas an RBF network needs a short training stage and then gives fast, smooth and compact classification.

4: CASE-BASED LEARNING (CBL) VS LOCALLY WEIGHTED REGRESSION (LWR) – RECOMMENDATION SYSTEMS

Introduction:
Both are lazy, memory-based approaches that postpone the work until a query arrives. CBL retrieves similar stored cases and reuses (and adapts) their solution, while LWR uses nearby stored examples to fit a local weighted regression model and compute a numeric prediction.

Example:
Example – movie recommendation: in CBL, each user's profile (genres liked, ratings, age, viewing history) with the movies that satisfied them is stored as a case; for a new user, the most similar earlier users are retrieved, their liked movies are reused, adjusted (e.g. removing movies already watched) and the outcome (clicked / rated) is retained. In LWR, to predict the rating a user would give to a movie, the model gives high weights to the ratings of similar users / similar movies (kernel on user features) and fits a local linear model to those ratings to estimate the rating.

Comparison:

Aspect	Case-Based Learning	Locally Weighted Regression
Type of output	A solution (list of recommended items) adapted from similar cases.	A numeric prediction (e.g. predicted rating) from a local regression.
Handling new data	New cases are retained after validation (Retain step); the case base is also pruned of redundant or outdated cases.	New data points are just added to the training set and are used at once; no retention logic is needed.
Adaptability	High: the retrieved solution is revised for the new situation, symbolic and structured information can be used, and similarity weights can be adjusted.	Adapts automatically to local trends in the data through the kernel weights; limited to numeric inputs and a regression form; τ controls the degree of adaptation.
Computational efficiency	Retrieval requires comparing the query with cases (reduced by indexing); adaptation adds extra effort; memory is large because full cases are stored.	Needs a weighted least-squares solution (matrix inversion) for each query – costly for large data sets; reduced by using only the nearest neighbours and approximate search.

Interpretability	High – the similar past case explains the recommendation.	Moderate – local coefficients and weights show the influence of neighbours.

Analysis:

- CBL is more suitable when recommendations must be explained, items have rich descriptions or rules, and a ready solution is required (cold-start can be solved by matching on profile data).
- LWR is more suitable when the target is a numeric score and the relationship between features and rating is smooth but non-linear.

Conclusion:

CBL gives explainable, adaptable recommendations and learns by retaining cases; LWR gives accurate numeric local predictions and absorbs new data instantly but at a higher cost per query. A hybrid (retrieve cases, then refine the rating with LWR) is often effective.

5: DECISION TREE LEARNING VS K-NEAREST NEIGHBOUR (KNN) – CUSTOMER CHURN PREDICTION

Introduction:
A decision tree is an eager, rule-based learner that builds a tree of feature tests using information gain or the Gini index. KNN is a lazy, instance-based learner that classifies a customer by the labels of the K most similar stored customers.

Example:
Example – customer churn: a telecom company has data such as tenure, monthly charges, contract type, number of complaints and usage. A decision tree may learn rules such as 'Contract = month-to-month AND complaints > 2 → churn = Yes'. KNN predicts that a customer will churn if most of the K = 7 most similar customers (by scaled distance) churned.

Comparison:

Aspect	Decision Tree	KNN
Interpretability	Very high: the tree gives clear if–then rules that business managers can follow and use to plan retention actions.	Low to moderate: gives only the label and the similar customers; no global rules or feature importance.
Training time	Training needed: roughly $O(n \cdot d \cdot \log n)$ to grow the tree; prediction is very fast (depth of the tree).	No training time (lazy); prediction is slow, $O(n \cdot d)$ per query without indexing.
Handling of noisy data	A fully grown tree over-fits noise and outliers; pruning, maximum depth or a minimum leaf size is needed.	Sensitive to noise for small K; a larger K or distance-weighted voting averages out noise; irrelevant features harm the distance.
Decision-making process	Hierarchical: tests one feature at a time with axis-parallel splits, choosing the most informative feature first.	Uses all features together through a distance calculation and a vote among neighbours.
Large datasets	Scales well in prediction; training is manageable and can be parallelised; handles mixed numeric / categorical data and missing values.	Memory and prediction cost grow with n; needs KD-tree / approximate search; needs feature scaling and encoding.

Complex feature spaces	Selects relevant features automatically and ignores irrelevant ones; has difficulty with diagonal boundaries and feature interactions unless the tree is deep.	Suffers from the curse of dimensionality: in high dimensions distances become similar; needs feature selection or PCA.

Analysis:

- For churn prediction a decision tree is usually preferred when the company needs explainable rules and fast scoring of millions of customers; KNN is useful as a quick baseline or for finding 'customers like this one'.
- A single tree is unstable (small data changes alter the tree), so ensembles such as Random Forest are often used to improve accuracy; accuracy, precision, recall and F1-score should be compared because churners are usually the minority class.

Conclusion:

The decision tree offers interpretability and fast prediction at the cost of a training stage and a risk of over-fitting; KNN needs no training and is simple, but it is slow on large data, sensitive to irrelevant features and less explainable.